Project Plan: Feedback Intelligence Platform

Status	Approved
Version	1.8
Date	2026-11-05
Owner	Benjamin Rose

1. Purpose and goals

Purpose

Product teams receive more user feedback than they can read. The Feedback Intelligence Platform automatically collects public GitHub issues for selected open-source products, classifies and summarizes them with AI, and publishes a daily dashboard showing what users struggle with and what is trending.

Goals

1. Deliver a working, publicly viewable system that updates daily without manual effort.
2. Operate at $0 cost indefinitely, with no trials or subscriptions.
3. Follow a complete, documented SDLC so the engineering process is as visible as the code.
4. Demonstrate professional software engineering skills: C#, SQL Server, AI integration, testing, CI/CD, monitoring, and disciplined AI-assisted development.

2. Scope

In scope

- Collecting public GitHub issues from three open-source repositories (`microsoft/winget-cli`, `microsoft/PowerToys`, `microsoft/vscode`), incrementally, once a day. Repositories are onboarded one at a time, starting with `winget-cli`.
- Redacting personal information before AI processing or publishing.
- AI classification (category, severity), summaries, emerging topics, and a weekly digest.
- A SQL Server data pipeline that turns raw issues into reporting tables.
- A public, read-only dashboard hosted on GitHub Pages.
- Automated testing, CI/CD, data-quality checks, and failure alerts.
- Project documentation covering every SDLC phase.

Out of scope

- User accounts, logins, or personalized views.
- Writing anything back to GitHub (labels, comments).
- Real-time or streaming updates; data refreshes daily.
- A live AI chat or question-answering feature.
- Feedback sources other than GitHub issues (possible future work).
- Paid cloud hosting or any service that requires a trial or subscription.

3. Stakeholders

Stakeholder	Role	What they care about
Project owner	Engineer, product owner, and operator	Building it well, keeping it running, $0 cost
Product teams (target users)	Read the dashboard to understand user pain points	Accurate categories, clear trends, data that's current
Technical reviewers (hiring managers, interviewers)	Evaluate the system and the process behind it	Code quality, design decisions, testing, documentation
Open-source communities (issue authors and maintainers of the three repositories)	Source of the data	Their content used respectfully; personal information not republished
Platform providers (GitHub: API, Actions, Models, Pages)	Supply the free services the system runs on	Usage within rate limits and terms of service

Issue authors never use the system, but their writing is what it processes. Protecting their personal information is therefore a requirement, not an option.

4. Constraints

Budget

- $0, permanently. No paid services, trials, subscriptions, or credits.
- No payment method on file with GitHub, so nothing can be charged by accident.

Time

- About 5 hours per week, with no fixed deadline.
- Work is planned in small, self-contained steps that fit a 1–2 hour session.

Team

- One person fills every role: engineer, product owner, tester, and operator.
- No peer reviewers; all reviews are self-reviews against a written checklist.

Technology

- Application code in C# (.NET 10; see ADR 0014).
- Data storage and transforms in SQL Server (Express in the pipeline, Developer locally).
- Hosting, scheduling, CI/CD, and alerting on GitHub (Actions, Pages, Issues).
- AI through GitHub Models only.

Platform limits

- GitHub Models free tier: roughly 15 requests per minute and 150 per day for lower-tier models, subject to change by GitHub.
- GitHub Copilot Free: 2,000 code completions and 50 chat requests per month.
- GitHub API rate limits for fetching issues.
- GitHub Actions scheduled workflows pause after about 60 days without repository activity.
- SQL Server Express: 10 GB maximum database size.

Legal, licensing, and privacy

- SQL Server Developer edition may only be used for development and testing, never in the production pipeline.
- Use of GitHub data and services must follow GitHub's terms of service.
- No personal information from issues may be sent to the AI or published.

5. Assumptions

#	Assumption	How we'll validate it	When	Result
A1	GitHub Models will remain free, with rate limits close to today's	Check GitHub's docs; monitor rate-limit responses in the pipeline	Before AI work; ongoing	—
A2	GitHub's Issues API stays publicly accessible, and its rate limits suit daily incremental fetches	Make test calls against winget-cli and measure calls used per run	Walking skeleton	Confirmed in M2: a 90-day fetch for winget-cli finished in under 4 seconds, far inside the limits
A3	GitHub Actions and Pages stay free for public repositories	Check GitHub's docs and billing page	Before deployment	Confirmed in M2: October's usage was fully covered by included usage, so the net cost was $0
A4	SQL Server Express runs in a Docker container on GitHub's Linux runners within reasonable time	Run a test workflow that starts SQL Server and runs one query	Walking skeleton	Confirmed in M2: SQL Server was ready before the pipeline started (migration took 0.7 s); the whole run took 49 s
A5	Presidio detects the common PII in issues (emails, IPs, tokens, usernames in file paths) well enough	Test against a hand-checked sample of real issues	Before redaction is enabled	Confirmed for titles at the end of M3: the 216 published winget-cli titles contained no emails, IP addresses, tokens, or usernames in paths; 3 had a four-part version number (such as 7.6.8.0) redacted as an IP address. Over-redaction is the safe failure, so it's accepted. Bodies are checked again in M4, before they reach the AI
A6	The AI model classifies issues accurately enough to be useful	Measure against a hand-labeled set of about 100 issues	Before AI results are published	—
A7	winget-cli's issue volume fits within the free AI limits, including the first backfill	Count issues over the planned history window; estimate requests needed	Before first AI run	—

#	Assumption	How we'll validate it	When	Result
A8	Analyzing public issues and showing summaries with links to the originals is permitted	Review GitHub's terms of service and acceptable use policies	Before issue titles or summaries are published (M3)	Reviewed 2026-11-01: GitHub's Acceptable Use Policies allow using public information collected through the API, and prohibit spam, selling personal information, and excessive automated activity. The project uses the API within rate limits, stores no usernames, redacts personal information, publishes only redacted titles and summaries with links to the originals, and removes issues on request (see README). This is a reading of the policies, not legal advice
A9	A Windows PC running WSL 2 (Ubuntu) and Docker Desktop can run SQL Server and Presidio containers for local development, with VS Code connected to WSL	Install WSL, Docker Desktop, and VS Code; start both containers and connect from SSMS	Before development starts	Confirmed in M1

6. Risks

Likelihood and impact are rated Low, Medium, or High.

#	Risk	Likelihood	Impact	Mitigation	Contingency
R1	GitHub Models changes its free limits or ends the free tier	**Medium**	**High**	AI provider sits behind a C# interface; cache all results so nothing is reprocessed	Switch to another free provider or a local model; dashboard still works without new AI results
R2	AI request volume exceeds the free daily limit (especially VS Code and first backfills)	**High**	**Medium**	Batch several issues per request; cap backfill to a history window; onboard repositories one at a time	Spread backfill across several days; process newest issues first
R3	AI classifications are inaccurate	**Medium**	**Medium**	Hand-labeled evaluation set; CI blocks prompt changes that lower accuracy	Show category as "AI-suggested"; tune prompts or narrow categories
R4	Prompt injection: issue text contains instructions that manipulate the AI's output	**Medium**	**Medium**	Treat issue text strictly as data; require structured JSON output and validate it; the AI has no ability to take actions	Reject invalid output and mark the issue "unclassified"

#	Risk	Likelihood	Impact	Mitigation	Contingency
R5	Personal information leaks to the AI or the public dashboard	Low	High	Presidio redaction before any AI call; no usernames stored; tests with known PII samples	Remove the data, republish, add the missed pattern, write a postmortem
R6	Scheduled workflow disabled after ~60 days without repository activity	High	Low	Pipeline commits its outputs daily; check before interviews	Re-enable with one click; dashboard keeps showing last good data
R7	GitHub API changes or returns unexpected data	Low	Medium	Schema validation on ingest; versioned migrations	Quality checks stop the run and open an alert issue
R8	SQL Server container is slow or fails on GitHub runners	Low	Medium	Validate early in the walking skeleton (A4)	Retry step; health-check wait before queries
R9	Repository grows too large from committing data history daily	Medium	Medium	Store compact Parquet; keep a limited history window; monitor repository size	Move history to GitHub Releases assets or prune old snapshots
R10	Accidental charges	Low	Medium	No payment method on GitHub; paid usage left off	Check billing settings monthly
R11	Secrets leaked in code or logs	Low	High	Use only the built-in `GITHUB_TOKEN`; secret scan in CI on every pull request; GitHub push protection	Revoke and rotate immediately; audit history
R12	Scope creep or lost momentum at 5 hours a week	Medium	High	Fixed scope; small steps; project presentable at every milestone	Ship the current milestone and pause new features

Review at the end of M3 (2026-11-05):

- **R6:** the pipeline has committed history to the `data` branch every day since 2026-11-01. Whether those commits count as activity can only be seen after 60 days, so it's checked again in M7, before interviews.
- **R9:** each day's history is about 37 KB of Parquet files, so a year of daily commits for winget-cli adds roughly 13 MB. That's well within GitHub's limits; it's measured again in M6, when larger repositories are added.
- **R5:** covered in M3 by redaction before anything is stored, the PII test set run end to end in CI and against the live service on every run, and the data contract test. See the test strategy.

7. Milestones and timeline

Estimates assume about 5 hours per week, with code drafted by an AI assistant and reviewed by the project owner. Each milestone ends with the project in a presentable state. Estimates are revisited at

every milestone, and actual time is recorded against them in the **Actual** column.

Milestone	What's delivered	Est. weeks	Actual	Presentable as
M0: Plan and design	Project plan, requirements, design docs, ADRs	1	6 days (Oct 13–18)	A well-documented project plan
M1: Foundation	Dev environment (WSL, Docker), repository, standards, PR template, CI that builds and runs tests	1	3 days (Oct 18–20)	A professionally set-up repository
M2: Walking skeleton	winget-cli issues → SQL Server in Actions → one number on GitHub Pages, all automated	1	5 days (Oct 20–24)	A working end-to-end pipeline, however small
M3: Core data pipeline and privacy	Incremental ingest, Presidio redaction and PII tests, raw → cleaned → reporting T-SQL, quality checks, alert issues, config validation in CI, test strategy document, basic dashboard (overview, trends, issue explorer)	2	12 days (Oct 25–Nov 5)	A complete, privacy-safe data engineering project
M4: AI enrichment	AI classification and summaries, caching, evaluation set with CI gate	1–2	—	An AI-integrated system with responsible-AI practices
M5: Insights	Emerging topics, weekly digest, pipeline health page	1	—	The full product vision for one repository
M6: Scale out	Onboard PowerToys, then VS Code with batching	1	—	A system that scales across products
M7: Operate and polish	Runbook, simulated incident and postmortem, Bicep templates, final README, walkthrough video	1	—	Portfolio-ready
M8: Could-haves *(optional)*	CSV export, Power BI Desktop report, local Spark partitioning prototype	Not estimated	—	Extended into Power Platform and Spark

Total for M0–M7: roughly 9–10 weeks. Building finishes sooner; the remaining time covers review and lets the pipeline accumulate real operating history. M8 starts only after M0–M7 are complete.

M2's daily schedule (FR-OPS-01) counted as done only once a scheduled run, not just a manual one, had succeeded. The first scheduled run succeeded on 2026-10-26, followed by daily successes through 2026-11-01, so M2 is complete.

Personal information is redacted from M3 onward, before any issue text is published. From M2, usernames are never stored and raw text is never committed.

8. Success criteria

Goal 1: A working, public system that updates daily without manual effort

- The dashboard is publicly reachable at a GitHub Pages URL with no login.
- The pipeline runs on schedule and succeeds on at least 95% of days over any 30-day period.
- Dashboard data is never more than 48 hours old, unless an alert issue has been opened explaining why.
- Every pipeline failure opens an alert issue automatically.
- All three repositories are onboarded.

Goal 2: $0 cost indefinitely

- Total spend is $0, confirmed by checking GitHub billing monthly.
- No payment method is on file with any service used.
- AI usage stays within the free daily limit, with no rate-limit failures over 30 days.

Goal 3: A complete, documented SDLC

- Every SDLC phase has its document in the repository: plan, requirements, design and ADRs, test strategy, runbook, and at least one postmortem.
- Every change goes through a pull request with a completed self-review checklist.
- Every pull request includes the AI-disclosure section.

Goal 4: Demonstrate professional engineering skills

- Automated test coverage of at least 80% for C# code.
- Every SQL transform has an expected-output test.
- AI classification accuracy of at least 80% on the hand-labeled evaluation set, enforced in CI.
- Zero known PII in published data, verified by tests with known PII samples.
- A newcomer can run the system locally from the README in under 30 minutes.

Revision history

Version	Date	Change
1.8	2026-11-05	Updated the R11 mitigation after adding a secret scan to CI.

Version	Date	Change
1.7	2026-11-05	Completed M3: recorded its actual time, the result of assumption A5, and a review of risks R5, R6, and R9; linked the new test strategy; corrected the secret-scanning description.
1.6	2026-11-01	Recorded the review of assumption A8 before issue titles are published.
1.5	2026-11-01	Recorded that M2's daily schedule was confirmed by scheduled runs from 2026-10-26.
1.4	2026-10-31	Added config validation in CI (FR-CFG-02) and the test strategy document to M3; noted that M2 needs a successful scheduled run.
1.3	2026-10-24	Recorded actual time for M0–M2 and results for validated assumptions (A2, A3, A4, A9). A8 now applies before issue titles are published, since the M2 dashboard shows only counts.
1.2	2026-10-19	Updated the runtime constraint from .NET 8 to .NET 10, because .NET 8 support ends in November 2026 (ADR 0014).
1.1	2026-10-14	Moved PII redaction from M4 to M3 so no issue text is published unredacted; renamed M3 and M4; added optional M8 for Could-have requirements. See requirements.
1.0	2026-10-14	Initial plan approved